

Parametro de configuracion de herramientas utilizadas (valores y ajustes especificos que determinan su rendimiento comportamiento y funcionamiento).

1. Backend (Node.js + Express)

- **Autenticación y Seguridad (JWT):**
 - **Claves Secretas:** Se configurará una clave secreta robusta para la firma de tokens JWT, gestionada a través del patrón Singleton para asegurar una única instancia segura en toda la aplicación.
 - **Tiempos de Expiración:** Se establecerán tiempos de expiración definidos para los tokens, forzando a los administradores a reautenticarse periódicamente para mantener la seguridad.
 - **Rendimiento y Escalabilidad:**
 - **Gestión de Conexiones:** Se implementará un pool de conexiones a MongoDB Atlas para reutilizar conexiones y reducir la latencia.
 - **Manejo de Concurrencia:** Se optimizará para gestionar y procesar datos de un mínimo de **10,000 dispositivos IoT concurrentemente**.
-

2. Base de Datos (MongoDB Atlas)

- **Optimización de Consultas:**
 - **Estrategia de Indexación:** Se crearán índices en las colecciones sobre los campos utilizados para filtros frecuentes (como fecha, zona geográfica o ID de luminaria) para acelerar las consultas y mejorar los tiempos de respuesta.
 - **Respaldo:**
 - **Copias de Seguridad:** Se activará una política de **respaldo automático semanal** de la base de datos para prevenir la pérdida de información ante cualquier eventualidad.
-

3. Frontend (React.js)

- **Rendimiento de Carga:**
 - **Optimización de Activos:** Se configurarán herramientas como Vite para aplicar técnicas de *code splitting* (división de código) y *lazy loading* (carga perezosa) de componentes, con el objetivo de que el dashboard principal se cargue en **menos de 2 segundos**.
 - **Integración de Servicios:**
 - **Mapas (Leaflet.js):** Se configurará la librería para utilizar capas de OpenStreetMap, mostrando marcadores interactivos que representen la ubicación y el estado de cada luminaria.
 - **Gráficas (Chart.js):** Se definirán los tipos de gráficos (líneas, barras, etc.) para la visualización de datos históricos y en tiempo real del consumo energético, conectándolos directamente a los endpoints de la API.
-

Pruebas Manuales Específicas

Esta fase es crucial para validar la experiencia de usuario, la seguridad en escenarios reales y el comportamiento del sistema con datos complejos.

Frontend y Visualización

- **Pruebas de Responsividad y Compatibilidad:**

- Verificar manualmente que el dashboard web se adapte y funcione correctamente en diferentes tamaños de pantalla y en las últimas versiones de los navegadores

Chrome, Firefox y Edge.

- **Pruebas de Visualización de Datos:**

- Confirmar visualmente que los mapas interactivos de

Leaflet.js muestren la ubicación y estado de las luminarias de forma precisa.

- Asegurar que las gráficas de

Chart.js representen correctamente los datos históricos y en tiempo real del consumo energético.

- **Pruebas de Carga de Datos y Velocidad:**

- Medir manualmente el tiempo de carga del dashboard para asegurar que sea

inferior a 2 segundos en condiciones de red estándar.

- Validar que los datos se carguen en la interfaz de forma correcta y sin demoras excesivas al aplicar filtros por fecha, zona o luminaria.

API, Base de Datos y Carga Masiva

- **Pruebas de Carga de Datos Masivos:**

- **MongoDB Atlas** y verificar que el sistema los procese sin errores ni degradación del rendimiento.

- **Pruebas de Integridad en Base de Datos:**

- Realizar consultas manuales a la base de datos para confirmar que la información recibida a través de la API se almacena de manera correcta, consistente y sin corrupción.

Seguridad Manual

- **Pruebas de Autenticación y Autorización (API):**
 - Utilizando Postman, se realizarán peticiones a los endpoints protegidos:
 - Con un **token JWT válido** (Bearer Token) para confirmar el acceso.
 - Sin token o con un token inválido para asegurar que se deniegue el acceso.
 - Con tokens de diferentes roles de usuario para verificar que los permisos se apliquen correctamente.
- **Pruebas de Expiración de Token:**
 - Se comprobará manualmente que, una vez que el token JWT expira, el sistema cierre la sesión del usuario o le solicite volver a iniciar sesión para continuar.

Dispositivos Wearable (Smartwatch)

- **Pruebas de Responsividad y UI:**
 - Verificar manualmente en dispositivos

Wear OS 3+ (o emuladores) que la interfaz de la aplicación se visualice correctamente y que todos los elementos sean funcionales en una pantalla de tamaño reducido.

- **Pruebas Funcionales:**
 - Confirmar que la aplicación del smartwatch se conecte a la API y muestre con precisión el consumo energético y el estado operativo de las luminarias.

Pruebas Automatizadas

Esta fase se centra en la validación continua y automatizada de la lógica del sistema.

- **Pruebas de Integración:** Se uso **Postman** para verificar que la comunicación entre el frontend, la API y la base de datos sea correcta.
- **Pruebas Unitarias:** Se validará el funcionamiento aislado de cada componente de **React (con Jest)** y la lógica de negocio en **Kotlin (con JUnit)**.
- **Pruebas End-to-End (E2E):** Mediante **Cypress**, se automatizarán flujos de usuario completos, desde el login en la web hasta la visualización de datos, para asegurar la integridad de la experiencia.

Caso de Prueba 1: Seguridad (Autenticación Exitosa)

- **ID del Caso de Prueba:** CP-SEC-001
- **Título:** Login de usuario con credenciales correctas.
- **Módulo:** Autenticación (RF-11).
- **Precondiciones:**
 - La API está en ejecución y conectada a la base de datos.
 - Existe un usuario registrado con el correo

angel.lopez@urbana.com y la contraseña Admin123!.

- **Pasos a Seguir:**
 - Realizar una petición

POST al endpoint /api/auth/login.

- En el cuerpo (body) de la petición, enviar el JSON con el correo y la contraseña del usuario.

- **Resultado Esperado:**
 - El sistema responde con un código de estado

200 OK.

- La respuesta JSON contiene

"success": true , un

token JWT , y los datos del usuario como
usuario id, rol y nombre.

Caso de Prueba 2: API para Frontend (Obtener Datos para el Mapa)

- **ID del Caso de Prueba:** CP-API-001
- **Título:** Obtener todas las luminarias activas para visualización en el mapa.
- **Módulo:** Gestión de Luminarias (RF-02).
- **Precondiciones:**
 - El usuario está autenticado y posee un token JWT válido.
 - Existen luminarias en la base de datos con el campo

activo: true.

- **Pasos a Seguir:**
 - Realizar una petición

GET al endpoint /api/luminarias.

- Incluir en la cabecera (header) el campo

Authorization: Bearer <TOKEN VALIDO>.

- **Resultado Esperado:**
 - El sistema responde con un código de estado

200 OK.

- La respuesta es un arreglo de objetos JSON, donde cada objeto representa una luminaria y contiene los campos

id, identificador, coordenadas (lat, lng) y activo: true.

Caso de Prueba 3: API y BD (Recepción de Datos IoT)

- **ID del Caso de Prueba:** CP-API-003
- **Título:** Crear un nuevo registro de consumo de sensor.
- **Módulo:** Recepción de Datos en Tiempo Real (IoT) (RF-05).
- **Precondiciones:**
 - La API está en ejecución y conectada a la base de datos.
 - Se cuenta con un token de autenticación JWT válido.
 - Existe una luminaria con el ID

507f1f77bcf86cd799439012.

- **Pasos a Seguir:**

- Realizar una petición

POST al endpoint `/api/consumo`.

- En el cuerpo de la petición, enviar un objeto JSON con los datos del sensor: `luminaria_id`, `timestamp`, `consumo`, `lumenes` y `encendida`.

- **Resultado Esperado:**

- El sistema responde con un código de estado

201 Created.

- La respuesta JSON contiene el mensaje

"Registro de sensor creado exitosamente" y el id del nuevo registro creado en la base de datos.

Caso de Prueba 4: API para Wearable (Obtener Estadísticas)

- **ID del Caso de Prueba:** CP-WEAR-002
- **Título:** Obtener estadísticas de consumo de una luminaria específica.
- **Módulo:** Visualización de Datos Energéticos (RF-03).
- **Precondiciones:**
 - La API está en ejecución y conectada a la base de datos.
 - El usuario está autenticado con un token JWT válido.
 - Existe la luminaria con el ID

507f1f77bcf86cd799439012 y tiene registros de consumo asociados.

- **Pasos a Seguir:**

- Realizar una petición

GET al endpoint `/api/consumo/estadisticas/507f1f77bcf86cd799439012`.

- Incluir el token de autenticación en la cabecera

`Authorization`.

- **Resultado Esperado:**

- El sistema responde con un código de estado

200 OK.

- La respuesta es un objeto JSON que contiene las estadísticas agregadas como `consumoTotal`, `consumoPromedio`, `tiempoEncendida` y `totalRegistros` para la luminaria solicitada.

Caso de Prueba 5: Gestión de Usuarios (Actualización)

- **ID del Caso de Prueba:** CP-USER-002

- **Título:** Actualizar los datos de un usuario existente por su ID.

- **Módulo:** Gestión de Usuarios y Roles (RF-10).

- **Precondiciones:**

- La API está en ejecución y conectada a la base de datos.
- El usuario está autenticado con un token JWT válido y tiene rol de administrador.
- Existe un usuario con el ID

507f1f77bcf86cd799439011.

- **Pasos a Seguir:**

- Realizar una petición

PUT al endpoint `/api/usuarios/507f1f77bcf86cd799439011`.

- En el cuerpo de la petición, enviar un objeto JSON con los nuevos datos para el usuario (ej.

`nombre, apellido, rol, etc.`).

- **Resultado Esperado:**

- El sistema responde con un código de estado

200 OK.

- La respuesta JSON contiene

"success": true, el mensaje "Usuario actualizado exitosamente" y el objeto del usuario con los datos ya modificados.

Flujo de Trabajo con GitLab (Basado en GitHub Flow)

Este flujo se caracteriza por su agilidad y enfoque en la integración y el despliegue continuo. La idea central es que la rama principal (`main`) siempre debe estar en un estado funcional y lista para ser desplegada.

Razones de la Elección

- **Simplicidad y Agilidad:** Es un modelo sencillo que evita la complejidad de gestionar múltiples ramas de larga duración como `develop` o `release`.
- **Integración Continua:** Fomenta la integración de cambios de manera frecuente, lo que reduce conflictos y facilita la detección temprana de errores.
- **Revisión Temprana y Colaboración:** El uso de *Merge Requests* (equivalente a los *Pull Requests* en GitHub) es central, promoviendo la revisión de código y el feedback constante entre el equipo.
- **Despliegue Continuo:** Al mantener la rama `main` siempre estable, se facilita la automatización para desplegar actualizaciones tan pronto como se aprueban los cambios.

Pasos del Flujo de Trabajo

1. **Todo comienza desde `main`:** La rama `main` es la fuente de la verdad y refleja la versión más reciente y estable del proyecto.
2. **Creación de una Rama para cada Tarea (`feature branch`):**
 - Para cualquier cambio nuevo (una funcionalidad, un bug, etc.), se crea una nueva rama a partir de `main`.
 - El nombre de la rama debe ser descriptivo (ej. `feat/HU-05-agregar-filtro-fechas` o `fix/error-login-jwt`).
3. **Desarrollo y Commits Locales:**
 - El desarrollador trabaja en su rama local, realizando commits atómicos y descriptivos.
 - Se recomienda hacer `push` de la rama al repositorio remoto de GitLab frecuentemente para mantener una copia de seguridad y visibilidad del progreso.

4. Apertura de un *Merge Request* (MR):

- Una vez que el trabajo está listo para ser revisado (o incluso antes, para recibir feedback temprano), se abre un **Merge Request** en GitLab.
- El MR se crea para fusionar la rama de la tarea (`feat/HU-05...`) en la rama `main`.
- En la descripción del MR se explica **qué** hace el cambio, **por qué** se hizo y **cómo** se puede probar.

5. Revisión de Código y Discusión:

- Otros miembros del equipo revisan el código directamente en la interfaz del Merge Request de GitLab.
- Se realizan comentarios, se sugieren mejoras y se discuten los cambios. El pipeline de CI/CD configurado en GitLab ejecutará las pruebas automatizadas en este punto.

6. Fusión (*Merge*) a `main`:

- Una vez que el MR es aprobado por el equipo y todas las pruebas automatizadas pasan exitosamente, la rama se fusiona con `main`.
- GitLab ofrece la opción de "Squash and Merge" para combinar todos los commits de la rama en uno solo, manteniendo el historial de `main` más limpio.

Estrategia de Despliegue con GitLab CI/CD

La estrategia de despliegue se basa en un modelo de **Integración Continua y Despliegue Continuo (CI/CD)**, aprovechando las potentes herramientas nativas de GitLab para automatizar el proceso desde que el código es enviado al repositorio hasta que está en producción. Esto se integra perfectamente con el flujo de trabajo **GitHub Flow** (adaptado a GitLab).

1. Entornos de Trabajo

Se utilizarán dos entornos principales para asegurar la estabilidad, gestionados a través de la funcionalidad de *Environments* de GitLab:

- **Staging (Pruebas):** Un entorno idéntico a producción donde se despliegan automáticamente las nuevas funcionalidades después de ser fusionadas a la rama `main`. Aquí, el equipo puede realizar pruebas manuales y de usabilidad antes del lanzamiento final.
 - **Production (Producción):** El entorno en vivo que utilizan los usuarios finales. El despliegue a este entorno es un paso manual (o "manual job" en GitLab CI/CD) que se realiza solo después de que la versión ha sido validada en Staging.
-

2. Flujo del Despliegue Automatizado con GitLab CI/CD

El proceso se define en el archivo `.gitlab-ci.yml` y se activa cada vez que un desarrollador crea un *Merge Request* hacia la rama `main`.

1. **Desarrollo Local:** Un desarrollador crea una rama para una nueva funcionalidad (ej. `feat/grafica-consumo-historico`).
2. **Apertura del Merge Request:** Al terminar, abre un **Merge Request (MR)** en GitLab para fusionar su rama con `main`.
3. **Activación de la Integración Continua (CI):** La apertura del MR activa automáticamente el pipeline de **GitLab CI/CD**, el cual ejecuta los siguientes *jobs* (trabajos):
 - **Build (Construcción):** Compila el código y crea los artefactos necesarios para el despliegue.
 - **Test (Pruebas Automatizadas):** Se ejecuta la suite completa de pruebas:

- Pruebas Unitarias con Jest y JUnit.
 - Pruebas de Integración para la API.
4. **Revisión y Fusión:** Si todos los *jobs* del pipeline pasan con éxito, el equipo revisa el código en la interfaz del MR. Una vez aprobado, el MR se fusiona a la rama `main`.
 5. **Despliegue a Staging:** La fusión a `main` activa otro pipeline que ejecuta el *job* de despliegue al entorno de **Staging**.
 6. **Validación Manual en Staging:** El equipo realiza las pruebas manuales finales en el entorno de Staging, como las pruebas de usabilidad y la validación de los flujos de usuario completos.
 7. **Despliegue a Producción (Manual):** Una vez validada la versión en Staging, un miembro autorizado del equipo (como el líder del proyecto) activa manualmente el *job* de despliegue final al entorno de **Producción** desde la interfaz de GitLab. Este paso es deliberadamente manual para tener un control final y seguro sobre lo que se libera a los usuarios.